Lession 9 Exploratory Data Analysis

PAGE NO.....
DATE / /

Exploring the Product Table.

database: ~~product~~ adw
table-name: product.

product-id	product-name	product-size	product-category-id	product-qty-type
1	Apples	1 lb	1	lbs
2	Bananas	1 lb	1	lbs
3	Carrots	1 lb	1	lbs
4	Milk	1 gallon	2	Unit
5	Eggs	1 dozen	2	Unit
6	Breads	1 loaf	3	Unit
7	Croissant	1 piece	3	Unit
Int primary key	Varchar(100) table-name: product	Varchar(50) product-category.	Int	Varchar(10)

product-category-id	product-category-name
1	Fresh Fruits & Vegetables
2	Dairy & Eggs
3	Bakery
4	Freshly Prepared Food.
Int primary key	varchar(100)

```
Select
    pc.product-category-id,
    pc.product-category-name,
    Count(p.product-id) As counts-of-products.
From product-category As pc
Left Join product As p
    on pc.product-category-id = p.product
                                -category-id
Group by pc.product-category-id, pc.
         product-category-name
Order by pc.product-category-id;
```

This example strictly follows the source material and demonstrates how to explore and analyze data in the product and product-category tables using SQL.

PAGE NO.....
DATE / /

product_category_id	product_category_name	count_of_products
1	Fresh Fruits & Vegetables	3
2	Dairy & Eggs	2
3	Bakery	2
4	Freshly Prepared Food	0

dhawa dorje ghising

PAGE NO.....
DATE / /

Exploring Possible Column Values.

table_name = vendor_inventory

market_date	vendor_id	product_id	quantity	original_price
2023-04-03	7	4	30.00	4.00
2023-04-03	8	5	20.00	3.50
2023-04-10	7	4	25.00	4.00
2023-04-10	8	5	15.00	3.50
2023-04-17	7	4	35.00	4.00
2023-04-17	8	5	10.00	3.50
2023-04-30	4	6	50.00	2.00
2023-10-10	7	4	40.00	4.00
2023-10-10	8	5	5.00	3.50
Date	Int	Int	Decimal (10,2)	Decimal (10,2)

Primary key (market_date, vendor_id, product_id)

1) Preview the vendor_inventory table

```
Select * From vendor_inventory
Limit 10;
```

2) Check for Unique Combinations.

```
Select market_date, vendor_id, product_id, Count(*)
From vendor_inventory
Group by market_date, vendor_id, product_id
Having Count(*) > 1;
```

PAGE NO.....
DATE / /

3) Analyze Date Ranges.

```
Select Min (market_date), Max (market_date)
From vendor_inventory;
```

output

MIN (market_date)	Max (market_date)
2023-04-03	2023-10-10

4) Analyze Vendor Activity

```
Select
    vendor_id,
    MIN (market_date) AS First_market_date,
    MAX (market_date) AS Last_market_date,
From vendor_inventory
Group by vendor_id
Order by First_market_date, Last_market_date;
```

output

vendor_id	First_market_date	last_market_date
7	2023-04-03	2023-10-10
8	2023-04-03	2023-10-10
4	2023-09-30	2023-09-30

Dhaval Gonsalves

PAGE NO.....
DATE / /

Explanation of the output:

1. vendor 7
   - First market date: 2023-04-03
   - Last market date: 2023-10-10

2. vendor 8
   - First market date: 2023-04-03.
   - Last market date: 2023-10-10

3. vendor 4:
   - First and last market date: 2023-09-30 (only one market date recorded).

dhawa dorje gurung

PAGE NO.....
DATE / /

Exploring Changes over Time.

table_name: vendor_inventory_1

vendor_id	product_id	market_date	quantity_available	original_price
4	1	2023-03-10	30	5.00
4	1	2023-04-15	40	5.00
4	1	2023-10-06	45	5.00
5	5	2023-08-10	15	8.50
5	5	2023-09-15	10	8.50
7	2	2023-07-14	35	6.00
7	3	2023-07-21	20	7.00
7	4	2023-05-10	25	4.00
7	4	2023-06-12	30	4.00
Int	Int	Date	Int	Decimal (6,2)

primary key (vendor_id, product_id, market_date).

1) Analyze Vendor Participation by Years and Months.

```
Select
    Extract (YEAR FROM market_date) As market_year,
    Extract (MONTH FROM market_date) As market_month,
    COUNT (DISTINCT vendor_id) As vendors_with_inventory
From vendor_inventory_1
GROUP BY EXTRACT (YEAR FROM market_date),
         EXTRACT (MONTH FROM market_date)
ORDER BY EXTRACT (YEAR FROM market_date),
         EXTRACT (MONTH FROM market_date);
```

chaya done chasing

PAGE NO.....
DATE / /

1) Grouping Process (Group By market_year, market_month)

market_year	market_month
2023	3
2023	4
2023	5
2023	6
2023	7
2023	8
2023	9
2023	10

-> Group by groups rows with the same value into one output row.

2) Counting Distinct Vendors (Count(Distinct vendor_id)); within each group, the query identifies and counts unique vendor_id;

Year	Month	Unique Vendors (vendor_id)	Count
2023	3	4	1
2023	4	4	1
2023	5	7	1
2023	6	7	1
2023	7	7	1
2023	8	5	1
2023	9	5	1
2023	10	4	1

PAGE NO.....
DATE / /

output

market_year	market_month	vendors_with_inventory
2023	3	1
2023	4	1
2023	5	1
2023	6	1
2023	7	1
2023	8	1
2023	9	1
2023	10	1

2) Explore a Specific Vendor's Inventory

```
Select
From vendor_inventory
Where vendor_id = 7
Order By market_date, product_id;
```

output

market_date	vendor_id	product_id	quantity	original_price
2023-04-03	7	4	30.00	4.00
2023-04-10	7	4	25.00	4.00
2023-04-17	7	4	35.00	4.00
2023-10-10	7	4	40.00	4.00

dhaval bhole ghising

3) Identify Seasonal Vendors.

```
Select vendor_id,
       Count ( Distinct Extract ( Month From market_date))
                    As active_months.
From vendor_inventory1
Group by vendor_id
Order by active_months desc;
```

1) `Extract ( Month From market_date ):`
   ↳ Extract the month (values from 1 to 12) from the `market_date` column.

2) `Count ( Distinct Extract ( Month From market_date )) As active_months:`

   ↳ Counts the unique months in which each vendor has inventory.
   -> This helps to identify how many different month each vendor participated in the market.

3) Group by vendor_id;
   -> Groups the results by each vendor.

4) Order by active_month Dese!

PAGE NO.....
DATE / /

1. Extract Month for Each Row

vendor_id	market_date	Extracted Month
4	2023-03-10	3
4	2023-04-15	4
7	2023-05-10	5.
7	2023-06-12	6
7	2023-07-14	7
7	2023-07-21	7
5	2023-08-10	8
5	2023-09-15	9
4	2023-10-05	10.

2. Group by vendor_id and Count Distinct Months

vendor_id	active_months.
4	3
7	3
5	2

output

3. Sort to active_months in Descending Order

vendor_id	active_months
4	3
7	3
5	2

dhaval done chirag

PAGE NO.....
DATE / /

4) Detect Market Closures-

```
with month As (
    Select 1 as month UNION Select 2 UNION Select 3
    UNION Select 4 UNION Select 5 UNION Select 6
    UNION Select 7 UNION Select 8 UNION Select 9
    UNION Select 10 UNION Select 11 UNION Select 12
)
Select m.month, COUNT (v.market_date) as active_days.
From months m
Left Join vendor_inventory I on
    ON m.month = Extract ( MONTH FROM v.market_date)
Group by m.month
Order by m.month;
```

In SQL, [UNION] is used to combine the results of two or more [SELECT] statements into a single result set. It ensures that the values returned by each [SELECT] are distinct (no duplicates). Each [SELECT] statement within a UNION must return the same number of columns and the columns must have compatible data types.

→ It combines all the values (1, 2, 3 etc) into a single column (aliased as [month]), which will contain values from 1 to 12 representing the months of the year.

There is month only on "select 1 as month" after there is no month?

→ The [month] alias from the first [select] is automatically applied to all the subsequent [select] statements in the [UNION]. This is why you only need to define the alias once.

PAGE NO.....
DATE / /

Does SQL inherently associate the word "month" with 30 days or any specific duration?

→ Not exactly, it treats the value [month] simply as a label for the numbers (1 to 12 in this case). month is just a column name. you can explicitly ask for it using date function. like Extract(). Therefore SQL does NOT "know" months have 30 days unless you use date functions.

```
Left Join vendor_inventory v
    On m.month = EXTRACT ( MONTH FROM v.market_date).
```

→ [Extract (Month From v.market_date)] pulls the month from the [market_date] in the [vendor_inventory] table.

→ [Left Join] ensures that every month from 1 to 12 will be included, even if there are no matching records in the [vendor_inventory] table

Example: if [vendor_inventory] has these dates:

vendor_id	market_date
4	2023-03-10
7	2023-07-14
5	2023-09-15

The query matches:

month	market_date
3	2023-03-10
7	2023-07-14
9	2023-09-15
(other month)	Null.

what happens in the left Join?

month (From months)	market_date (From vendor_inventory)
1	NULL
2	NULL
3	2023-03-10
4	2023-04-15
5	Null
6	Null
7	2023-07-14
7	2023-07-21
8	2023-08-10
9	2023-09-15
10	Null
11	Null
12	Null

→ Now, Count (v.market_date) counts the no. of the non-null market_date values for each month.

output	month	Count(v.market_date) As active_days
	1	0
	2	0
	3	1
	4	1
	5	1
	6	1
	7	2
	8	1
	9	1
	10	1
	11	0
	12	0

PAGE NO.....
DATE / /

Exploring Inventory vs. Sales.

table_name : vendor_inventory

market_date	vendor_id	product_id	quantity	Original_price
2023-10-01	1	101	50	5.00
2023-10-01	1	102	30	3.50
2023-10-01	2	201	20	10.00
2023-10-02	1	101	40	5.00
2023-10-02	2	201	15	10.00
Date	Int	Int	Int	Decimal (10,2).

customer_purchases

market_date	vendor_id	product_id	quantity	cost_to_customer_per_qty
2023-10-01	1	101	10	6.00
2023-10-01	1	101	15	6.00
2023-10-01	1	102	5	4.00
2023-10-01	2	201	8	12.00
2023-10-02	1	101	20	6.00
2023-10-02	2	201	10	12.00
Date	Int	Int	Int	Decimal (10,2)

```
Select
    vi.market_date,
    vi.vendor_id,
    vi.product_id,
    vi.quantity As quantity_available,
    COALESCE (sales.quantity_sold, 0) As quantity_sold,
     vi.original_price
    COALESCE (sales.total_sales, 0) As total_sales
FROM
    vendor_inventory2 As vi
    LEFT JOIN (
        SELECT
            market_date,
            vendor_id,
            product_id,
            SUM (quantity) As quantity_sold,
            SUM (quantity * cost_to_customer_per_qty)
                                    As total_sales
        FROM
            customer_purchases
        GROUP BY
            market_date, vendor_id, product_id
        ) As sales
    on vi.market_date = sales.market_date
    AND vi.vendor_id = sales.vendor_id
    AND vi.product_id = sales.product_id
ORDER BY
    vi.market_date, vi.vendor_id, vi.product_id;
```

PAGE NO.....
DATE / /

Key differences

	Joining a Table Directly	Joining a Subquery
What is joined	An existing table in the table	The result of a query (a temporary table).
Syntax	Left Join table_name As alias	Left Join (subquery) As alias.
Use case	Simple joins where no aggregation or transformation is needed.	Complex joins where you need to aggregate, Filter, or transform data before joining.
Performance	Generally faster because it involves fewer steps	May be slower because the subquery must be executed first.
Flexibility	Limited to the structures of the existing table	Highly flexible: you can shape the data as needed.

PAGE NO.....
DATE / /

COALESCE

→ Replaces `NULL` values in quantity_sold and `total_sales` with 0 for cases where no sales were made.

Joining Subquery output:

market_date	vendor_id	product_id	quantity_sold	total_sales
▶ 2023-10-01	1	101	25	150.00
~~202~~				
2023-10-01	1	102	5	20.00
2023-10-01	2	201	8	96.00
2023-10-02	1	101	20	120.00
2023-10-02	2	201	10	120.00

dhawa dorje ghising

final-output

Market_date	vendor_id	product_id	quantity_available	quantity_sold	original_price	total_sales
2023-10-01	1	101	50	25	5.00	150.00
2023-10-01	1	102	30	5	3.50	20.00
2023-10-01	2	201	20	8	10.00	96.00
2023-10-02	1	101	40	20	5.00	120.00
2023-10-02	2	201	15	10	10.00	120.00

dhawa dorje

PAGE NO.
DATE / /